

Memoria Técnica Gestual



**UNIVERSIDAD
DE GRANADA**

Santiago López Cerro
Mario Lindez Martinez
Antonio Javier Rodríguez Romero
Diego Ortega Sanchez
Jaime Corzo Galdó

1. Índice

1. Índice.....	2
2. Software.....	3
2.1. Librerías Externas.....	3
2.2. Assets.....	3
2.3. Descripción de clases.....	7
2.4. Grafo de la Escena Principal (EscenaPlanetas).....	11
3. Gestos Utilizados y Exploración del Tótem.....	14
4. Reparto de Trabajo.....	15

2. Software

A continuación detallamos todos los elementos de Software utilizados en la implementación del tótem gestual para el Planetario “Más Allá del Cielo”, en concreto el situado en la sala con el mismo nombre y encargado de proveer con información general sobre todos los planetas del Sistema Solar y del museo.

Es importante destacar que el proyecto se ha realizado con el editor de *Unity* versión 6.2.6f2.

2.1. Librerías Externas

En cuanto a los paquetes externos utilizados, solo se ha requerido la instalación del paquete *UltraLeap Tracking* para el uso del dispositivo *UltraLeap 3Di*. En él, encontramos la librería necesaria para acceder, a través de sus clases y métodos que se detallarán en los apartados siguientes, a los datos provistos por el sensor mencionado previamente.

Es necesario también, para la utilización del dispositivo, la instalación de los *drivers* correspondientes en el ordenador de ejecución.

2.2. Assets

Contamos con los *assets* que incluye el paquete [Kenney UI Pack - Sci-Fi](#) para conseguir una estética más atractiva y coherente en el tótem.

De este obtendremos plantillas para la creación de paneles y botones con los que se construye la “parte 2D” de la *UI*.

En concreto, los utilizados son:

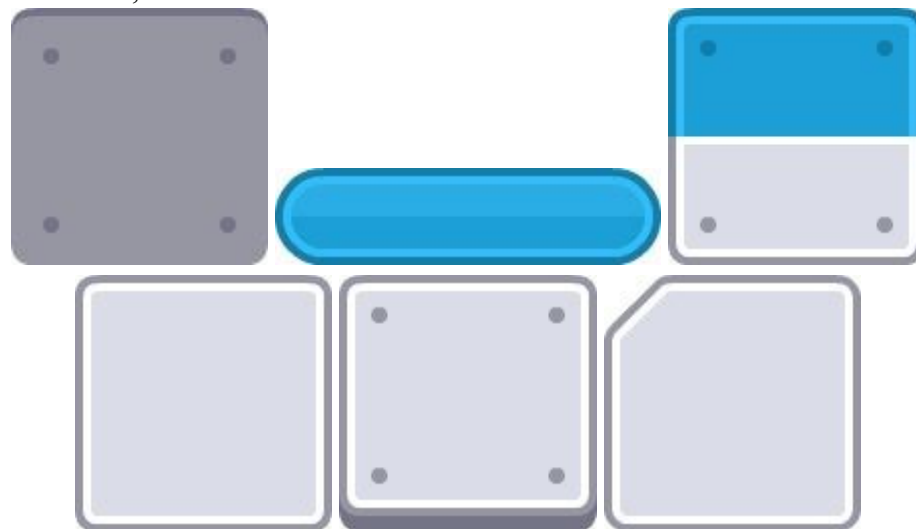


Figura 1: Assets utilizados del paquete UI Pack Sci-Fi

También se han usado algunos assets del paquete [2D Atlas Speech bubbles](#) [Alphabet Numbers](#) para dar un aspecto más atractivo a la respuesta por

parte de la UI que recibe el usuario al cambiar el modo de dificultad. El que se ha acabado utilizando es:



Figura 2: Imagen de Speech Bubble

Por otro lado, para la representación del Sistema Solar, hemos creado representaciones de cada uno de los planetas con esferas e imágenes en alta resolución como texturas de estas.

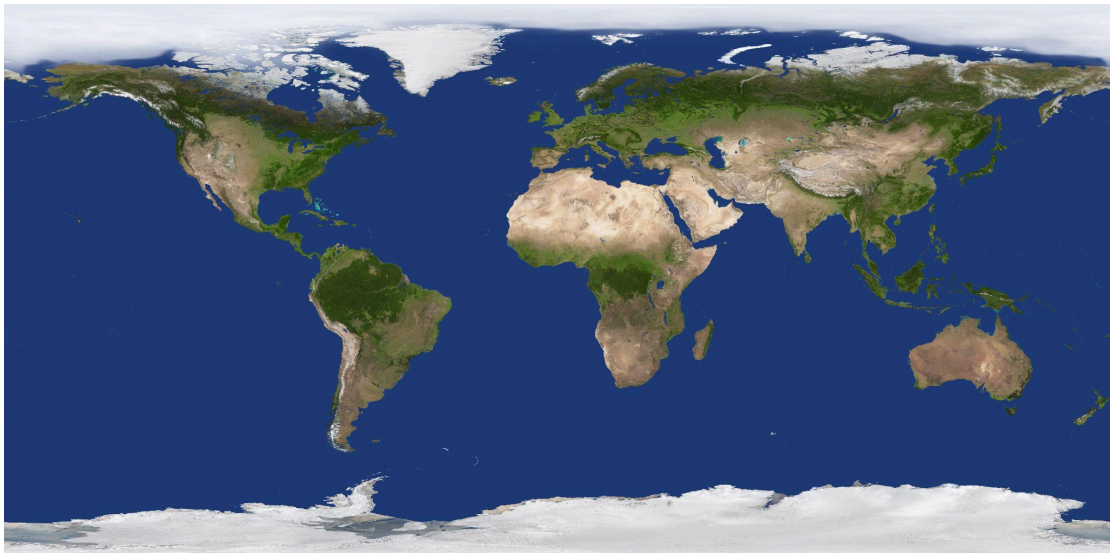


Figura 3: Imagen de la superficie de la Tierra en 8k

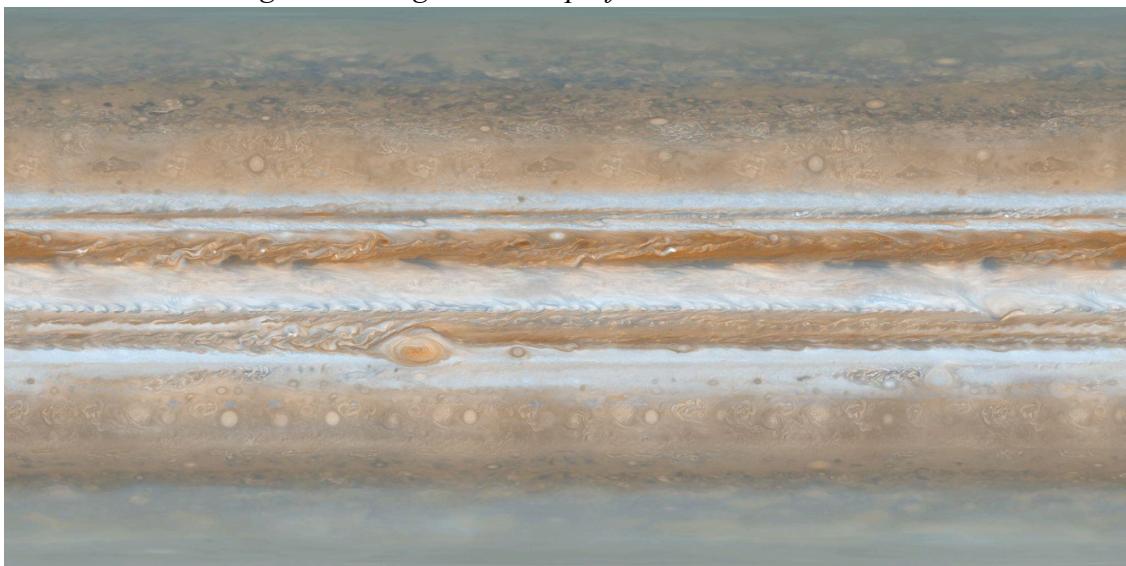


Figura 4: Imagen de la superficie de Júpiter en 8k

A su vez, para la representación 3D de las instalaciones, hemos utilizado el modelo y los planos, provistos por el docente, de la Escuela Técnica Superior de Ingeniería Informática y Telecomunicaciones de Granada.

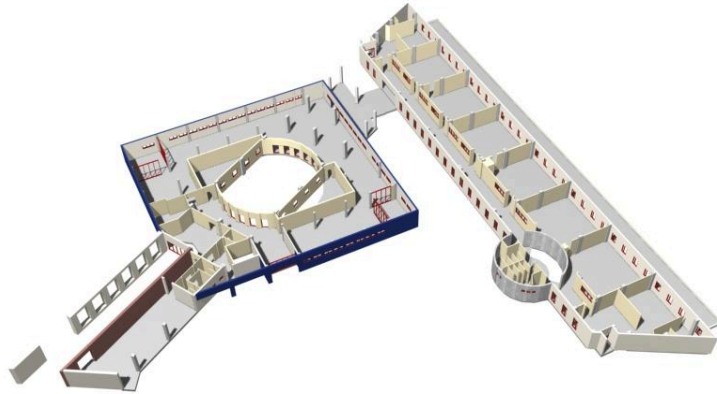


Figura 5: Planta primera ETSIIT

Para aportar un aire más lúdico y divertido, la selección de planeta cuenta con un cursor que representa la mano de un astronauta, el cual creamos con la ayuda de *ChatGPT* y formateamos para obtener en png.



Figura 6: cursor con forma de mano de astronauta

En la misma línea que el *asset* anterior, los distintos modos (Niños, normal y experto) se representarán con distintos dibujos de alienígenas

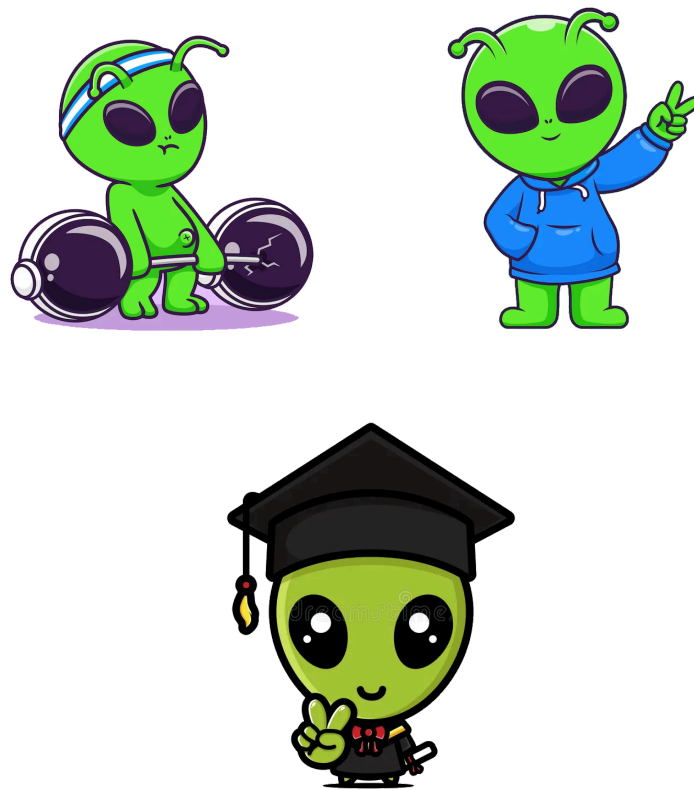


Figura 7: Alienígenas de los distintos modos

Por último, para añadir dinamismo al tótem, hemos incluido satélites que orbitan cerca de algunos planetas. Para ello, hemos utilizado los modelos que la misma NASA provee de estos. Los encontramos en la [web oficial de la NASA](#).



Figura 8: Captura de pantalla del modelo 3D del satélite Magellan

2.3. Descripción de clases

A continuación se describen las principales clases, tanto brindadas por las librerías estándar de *Unity* y del paquete *UltraLeap* y utilizadas para el acceso a la información del sensor como las creadas con el objetivo de administrar y gestionar los distintos menús y opciones de la UI gestual.

- *Hand*: Clase proporcionada por la librería de *UltraLeap* que representa una mano completa detectada en un frame. A través de este objeto podemos consultar de forma estructurada la postura y el movimiento de la mano. Además de exponer la posición y orientación de la palma (*PalmPosition*, *PalmNormal*, *Direction*) y su velocidad (*PalmVelocity*), la clase incluye una colección de dedos (*Fingers*), cada uno con sus propias articulaciones y posiciones, así como información global como la mano dominante (*IsLeft / IsRight*), la fuerza de agarre (*GrabStrength*) o el grado de apertura de los dedos (*PinchStrength*). Todo ello permite construir gestos complejos basados no solo en la presencia de la mano, sino también en su orientación, apertura, movimiento y relación entre dedos.
- *Vector3*: Estructura básica de la librería estándar de *Unity* que representa un vector o punto en el espacio 3D mediante tres componentes (x, y, z). Se utiliza de forma intensiva para trabajar con posiciones, direcciones y velocidades dentro de la escena.
- *PlanetZoomController*: Este componente está pensado para añadirse a un objeto de tipo *Camera*. Se encarga de gestionar la cámara cuando el usuario hace zoom sobre un planeta y al navegar entre planetas vecinos dentro de la vista del Sistema Solar. En el inicio guarda la posición y rotación por defecto de la cámara y mantiene un planeta objetivo (*currentTarget*) junto con una lista ordenada de planetas para poder moverse a sus vecinos. Cuando se solicita un zoom sobre un planeta, si no se está realizando ya otro movimiento, el controlador diferencia entre dos casos: si se pulsa de nuevo sobre el mismo planeta, ejecuta un zoom out devolviendo la cámara a su posición inicial; si se selecciona un planeta distinto, calcula una posición segura frente al planeta (en función de su tamaño y una distancia de zoom configurable), orienta la cámara hacia él y lanza una interpolación suave de posición y rotación (zoom in). Además, permite seleccionar automáticamente el planeta vecino (izquierda/derecha), con opción de volver al principio o al final de la lista, y al finalizar cada transición de cámara dispara el evento *OnZoomCompleted*, que notifica si la acción ha sido de acercamiento o alejamiento para que otros sistemas (como la interfaz del tótem) puedan reaccionar en consecuencia.
- *GameManager*: Se encarga de la escena principal. Es un singleton (hay uno solo) lo que permite que el resto de scripts pueden acceder a él con *GameManager.Instance*, también controla el modo de juego (*Kid/Normal/Expert*) y el flujo entre las distintas pantallas (*StandBy*, *Mainpanel*, vista de planetas y zoom a los planetas). Además coordina la UI mediante *RequestZoom* por ejemplo, para hacer click a un planeta o salir de él. Y otras funciones como activar/desactivar el blur de fondo al entrar o

salir de la visita, gestionar los tutoriales, responder a los gestos del swipe y por último permite ir a la escena donde se encuentra la maqueta del modelo 3D. De esta manera evitamos tener toda esta lógica repartida y duplicada en varios scripts.

- *UIManager*: UIManager es la clase encargada de toda la interfaz de usuario del planetario. Es un singleton accesible mediante UIManager.Instance y ofrece un sistema genérico de fades con CanvasGroup (ShowPanelFade) sobre el que se apoyan el resto de transiciones: entrada y salida del stand-by, aparición del menú inicial, apertura y cierre del panel de planeta y del panel de mapas, o la muestra del banner de modo.

Además, controla el menú de inicio (startMenuCanvas) y el panel de información de planeta (PlanetMenu), actualizando título y descripción según el planeta objetivo, el modo de juego y el idioma seleccionado, con los textos proporcionados por PlanetTextCSVLoader. A través de refreshUI se encargará de renovar los textos del menú principal (botones, título del planetario) y del panel de mapas. Por último, administra el panel de mapas del museo (mapsPanel) y los tutoriales en vídeo, mostrando u ocultando el canvas de tutoriales, reproduciendo el clip asociado a cada gesto y registrando cuál está activo (tutorialPlaying). Varios de sus métodos (como ClickedStart, ClickedMuseumMap o GoToMainPanel) se conectan directamente a los botones de la UI y delegan en el GameManager la lógica de navegación entre pantallas y escenas.

- *VideoManager*: Centraliza toda la lógica de reproducción de los vídeos de tutorial dentro de la escena: Es un singleton (VideoManager.Instance) para que cualquier script pueda lanzar o detener tutoriales. Gestiona dos VideoPlayer de Unity, asignando dinámicamente el VideoClip según el tipo de gesto y el idioma actual (click, swipe_der, swipe_izq, swipe_back). Mediante PlayTutorial(id) selecciona y reproduce el clip adecuado en función del tipo de tutorial (click, swipe lateral, swipe back). Con StopTutorial() detiene cualquier reproducción en curso. El SwipeTutorial() es un tutorial de swipe compuesto, ya que alterna en bucle los vídeos de deslizar a derecha e izquierda. Para decidir cuándo activar o silenciar el audio usa SetAudioMute, controlando así en qué repeticiones del vídeo se escucha sonido.
- *PlanetTextCSVLoader*: PlanetTextCSVLoader se encarga de la gestión de textos e idiomas del proyecto. Es un singleton que, si dontDestroyOnLoad está activo, se mantiene entre escenas para que todos los scripts compartan el mismo banco de textos. Al iniciarse, carga en memoria un CSV multi-idioma donde cada fila tiene un identificador (ID) y una columna por idioma (SPANISH, ENGLISH, etc.), y a partir de ahí construye un diccionario del tipo db[id][Language] = texto, además de una lista con los idiomas detectados. También fija un idioma por defecto (Spanish) e inicializa currentLanguage a ese valor.

Sobre esta base expone una API de consulta de textos, tanto genéricos como específicos de cada planeta y modo de juego. Además, permite cambiar el

idioma en tiempo de ejecución mediante `setLanguage(Language new_language)`, método que dispara el evento `languageChange` y que otros componentes, como `UIManager`, utilizan para refrescar automáticamente todos los textos cuando el usuario modifica el idioma.

- *PlanetClickable*: Implementada en “ClickablePlanet.cs”. Es la clase que hace que cada planeta del sistema sea “clicable” a través de eventos de UI. Implementa `IPointerClickHandler`, de modo que cuando se hace clic sobre el planeta (ya sea con ratón, Leap o cualquier sistema que dispare el evento), llama al `GameManager` mediante `GameManager.Instance.RequestZoom(this)` para iniciar el zoom hacia ese planeta concreto. Además, almacena datos asociados como la distancia de zoom recomendada (`zoomDistance`), un nombre visible (`displayName`) y un identificador interno (`_id_planeta`) accesible con `GetId()`, que sirve para enlazar el planeta con otras fuentes de datos, como los textos del CSV o información adicional.
- *SwipeGestureDetector*: Implementada en “Swipe.cs”. Es la clase que se encarga de detectar gestos de deslizamiento (swipe) con la mano derecha usando Leap Motion. Sirve para traducir movimientos rápidos de la mano en tres acciones gestuales principales (izquierda, derecha y hacia atrás) que el `GameManager` puede interpretar como órdenes de navegación dentro del tótem. Además usa un cooldown temporal para evitar múltiples detecciones consecutivas del mismo gesto.
- *LeapMousePointer*: Implementada en “FingerPointer.cs”, es la clase que traduce el movimiento de la mano derecha detectada por Leap Motion en un puntero de ratón dentro de la interfaz. Usa la posición de la palma para mover una imagen de cursor (`pointerUI`) sobre la pantalla, aplicando sensibilidad, inversión opcional de ejes y un suavizado para evitar movimientos bruscos. Cuando hay mano derecha visible, activa el puntero y lo mantiene totalmente opaco; cuando la mano desaparece, inicia un temporizador y hace un fade out progresivo mediante un `CanvasGroup` hasta ocultar de nuevo el cursor. De este modo, proporciona una forma visual y cómoda de señalar y seleccionar elementos en la UI usando gestos.
- *PoseGestureDetector*: Clase implementada en “ModeGesture.cs” que detecta poses de mano con Leap Motion para cambiar el modo de juego del planetario (Kid, Normal, Expert). Observa la mano (normalmente la izquierda), comprueba que esté abierta y bien orientada hacia la cámara y reconoce combinaciones de dedos extendidos que hemos definido como gestos de cambio de modo. Si el usuario mantiene una de estas poses el tiempo suficiente y ha pasado el cooldown desde la última detección, la clase avisa al `GameManager` para que actualice el modo de juego. Ofrecer una forma rápida y gestual de cambiar entre modos sin necesidad de botones físicos ni menús adicionales.
- *LeapPlanetSelector*: Implementada en “Pinching.cs”, es la clase que transforma el gesto de pinza (pinch) de la mano derecha, detectado por Leap Motion, en “clicks” dentro del sistema. Cada vez que se inicia un pinch

fuerte, primero intenta clicar sobre la interfaz (botones u otros elementos de UI) usando raycasting en pantalla y, si no hay ningún elemento de UI debajo del cursor, lanza un SphereCast en 3D para comprobar si se ha pinchado sobre un planeta (PlanetClickable) y seleccionarlo. Permite que el usuario pueda interactuar tanto con la interfaz gráfica como con los planetas 3D usando únicamente el gesto de pinza, sin necesidad de ratón ni pantalla táctil.

- *SceneAutoReset*: SceneAutoReset es la clase que vigila la presencia de manos en Leap Motion para gestionar la inactividad del tótem. En cada Update consulta el CurrentFrame del LeapServiceProvider: la primera vez que detecta manos avisa al GameManager para salir del estado de StandBy, y a partir de ahí, si dejan de verse manos, empieza a incrementar un temporizador (currentTimer). Cuando el tiempo sin manos supera el umbral configurado en timeToReset, registra un mensaje de inactividad y pide al GameManager que reinicie la escena, garantizando que la aplicación vuelva sola a su estado inicial después de un periodo prolongado sin uso.
- *ExitGesture*: Clase que detecta el gesto específico de salida con Leap Motion, basado en levantar ambas palmas hacia arriba. En cada Update comprueba que haya exactamente dos manos visibles y que la normal de cada palma esté orientada claramente hacia arriba; si la pose se mantiene de forma continua durante un tiempo mínimo (holdTimeParams), considera el gesto como válido y llama a SalirDeLaApp(), que cierra la aplicación (o detiene la ejecución en el editor). De esta forma implementa un gesto “oculto” y deliberado para apagar el tótem sin riesgo de activarlo por accidente.

2.4. Grafo de la Escena Principal (EscenaPlanetas)

En este apartado queda reflejada la estructura jerárquica de los diferentes objetos que componen la escena principal de nuestro proyecto de *Unity*.

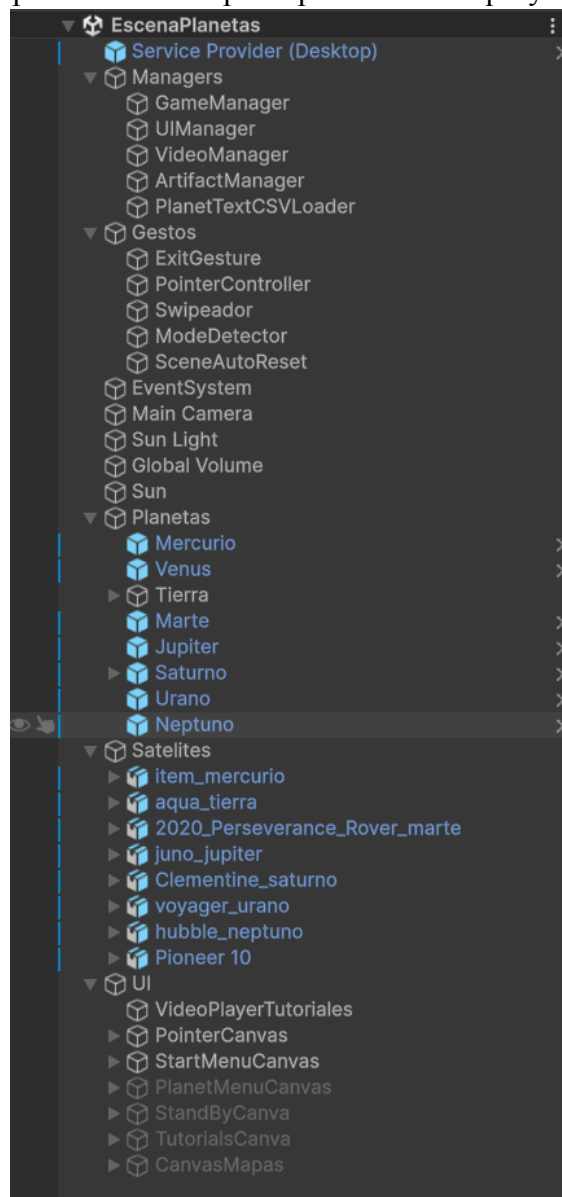


Figura 9: estructura en el editor de Unity de la escena

Como observamos en la figura 3, dentro de la escena principal, *EscenaPlanetas*, encontramos como elementos principales:

- *GameManager*: GameObject vacío en la escena que actúa como contenedor del componente GameManager y como punto central de control del estado global, los modos de juego y el flujo entre las distintas pantallas del planetario.
- *UIManager*: GameObject vacío en la escena que actúa como contenedor del componente UIManager y como raíz lógica de toda la interfaz de usuario del planetario.

- *VideoManager*: GameObject vacío en la escena que actúa como contenedor del componente VideoManager y como punto central desde el que se gestionan todos los vídeos de tutorial del planetario.
- *Service Provider*: Conecta el proyecto con el servicio de tracking de Ultraleap y entrega las manos ya transformadas al espacio de Unity. Todos los scripts que necesitan tracking leen los Frame actuales a través de este componente en lugar de comunicarse directamente con el servicio de Ultraleap.
- *Sun, Planetas* y los distintos satélites: Usados en la escena principal para reproducir un sistema planetario simplificado compuesto por un Sol central, varios planetas interactivos y distintos satélites o sondas asociados.
El Sol es el elemento central alrededor del cual se distribuyen los planetas, y sirve como ancla de composición de la escena 3D. Su función es principalmente estética y de referencia.

Cada planeta se implementa como un prefab 3D que combina geometría, materiales y componentes de interacción (por ejemplo, PlanetClickable). Estos objetos son utilizados por PlanetZoomController y GameManager para realizar los movimientos de zoom y navegación. Por otro lado, su identificador se emplea en PlanetTextCSVLoader para recuperar el nombre y la descripción del planeta en el idioma y modo de juego activos, que posteriormente se muestran en la interfaz (UIManager). De esta forma, los planetas constituyen los nodos principales de interacción del usuario dentro de la escena.

Adicionalmente se incluyen modelos 3D de satélites, sondas y otros elementos (por ejemplo, naves o rovers) asociados a determinados planetas. Estos objetos tienen una función principalmente informativa y de refuerzo visual: enriquecen la representación del sistema y permiten ilustrar misiones espaciales relacionadas. El objeto ArtifactManager se encarga de activarlos y desactivarlos según el planeta que se esté visitando.

- *UI*: Es un objeto vacío que actúa como raíz de toda la interfaz de usuario de la escena, tiene los siguientes elementos:
StartMenuCanvas: menú principal (título, botones de inicio y mapas, selección de idioma, botón de modo).
PlanetMenuCanvas: panel de información del planeta (título, texto descriptivo y cambio de idioma).
StandByCanva: pantalla de espera con los vídeos/elementos que se muestran antes de iniciar la visita.
VideoPlayerTutoriales y TutorialCanvas: soporte visual para los vídeos de ayuda.
PointerCanvas y Pointer: cursor controlado por Leap.
CanvasMapas / PanelMapas: panel para los mapas del museo.

Otros grupos como ModeBannerPanel, HandsPanel, ModeButton, etc.

En resumen, UI proporciona una estructura jerárquica única para localizar, habilitar y gestionar de forma coherente todos los elementos gráficos y paneles de la experiencia del planetario.

- *PlanetTextCSVLoader*: Se encarga de la gestión de textos e idiomas del proyecto. Es un GameObject vacío en la escena que actúa como contenedor del componente PlanetTextCSVLoader y como banco central de textos multi-idioma, accesible desde el resto de sistemas para obtener nombres, descripciones y cadenas traducidas.
- *PointerController*: Agrupa la lógica de “ratón por gestos” usando el Leap: Contiene el script Leap Mouse Pointer, que lee las manos del LeapServiceProvider, proyecta la posición del dedo a coordenadas de pantalla y mueve el Pointer como si fuera un cursor. Con FadeOutDelay oculta el cursor tras un tiempo sin detección de mano.

Incorpora además el script Leap Planet Selector, para detectar un gesto de pinch (umbral de “Pinch On/Off”) y lo interpreta como click.

Aplica un radio de tolerancia (Click Radius) para hacer el click más permisivo.

En conjunto, este objeto convierte los movimientos de la mano en un puntero de UI, que permite seleccionar botones o planetas sin un ratón físico.

- *Swipeador*: Es un objeto vacío que se encarga de la detección de gestos de swipe con la mano dominante (la derecha por defecto) usando el Leap. Contiene el script SwipeGestureDetector.
- *ModeDetector*: GameObject vacío en la escena que contiene el componente PoseGestureDetector encargado de detectar, mediante Leap Motion, las poses de mano asociadas al cambio de modo de juego (Kid / Normal / Expert) y de notificar estos cambios al GameManager.
- *ExitGesture*: GameObject vacío en la escena que contiene el componente ExitGesture, responsable de escuchar el gesto oculto de levantar ambas palmas hacia arriba con Leap Motion y, cuando se mantiene el tiempo requerido (establecido a 3 segundos), ordenar el cierre seguro de la aplicación del tótem.
- *SceneAutoReset*: GameObject vacío en la escena que contiene el componente SceneAutoReset, encargado de monitorizar la presencia de manos en Leap Motion, salir del estado de StandBy cuando se detectan por primera vez y reiniciar la escena automáticamente tras un tiempo configurado de inactividad.

- **Main Camera:** La Main Camera es la cámara principal de la escena y representa el punto de vista del usuario en el tótem, desde el que observa el Sistema Solar y cada uno de los planetas. Sobre esta cámara se añade el componente PlanetZoomController para que, cuando el usuario selecciona un planeta o navega a su vecino, se desplaza hasta una posición segura frente a ese cuerpo celeste y lo encuadra correctamente, ofreciendo una experiencia de exploración cómoda y controlada.
- **Sun Light:** Fuente de luz direccional principal de la escena. Simula la iluminación del “sol” sobre el Sistema Solar, definiendo la dirección, intensidad y color de la luz que reciben los planetas y el resto de elementos 3D, así como las sombras proyectadas.
- **Global Volume:** Volumen global de postprocesado (URP) que aplica efectos visuales al conjunto de la escena, como el desenfoque de fondo (Depth of Field) y otros ajustes de imagen. Sirve como punto central donde se configuran y controlan estos efectos, incluyendo el desenfoque que activa y desactiva el BlurVolume según el estado de la visita.

3. Gestos Utilizados y Exploración del Tótem

Hablando ahora explícitamente de la interacción con el tótem, como ya hemos descrito en la memoria del museo, el público encontrará el tótem implementado en la primera planta, en la sala “*Más Allá del Cielo*”, sala introductoria a todo el planetario y, más en concreto, al Sistema Solar.

Nuestro enfoque se ha centrado en crear un tótem que brinde información general sobre cada uno de los planetas, a la vez que el usuario visualiza y explora el Sistema Solar.

En un primer momento, el usuario encontrará un menú de inicio con el nombre del planetario (“*Planetario Más Allá del Cielo*”) y ciertas opciones de accesibilidad, como el idioma. Además, el tótem se adaptará al nivel del usuario, pudiendo activar tanto el modo normal como el infantil o el experto.

Como suponemos que puede ser poco intuitiva en ciertos casos una interfaz gestual, cada gesto vendrá acompañado de un vídeo explicativo del gesto que aparezca la primera vez que se tenga que realizar, con un audio en el idioma seleccionado.

Una vez seleccionado el idioma y, en el caso que corresponda activado el modo infantil, el usuario pasará a ver una vista cenital del Sistema Solar, con todos los planetas alineados. Este podrá recorrer uno por uno, ya sea seleccionando el que le interese o por orden, observando satélites que orbitan en la realidad a estos planetas y leyendo una pequeña descripción de cada uno de estos. Desde este menú del planeta, podrá tanto avanzar al siguiente como retroceder o volver a la vista cenital.

Pasando a enumerar los gestos que el usuario deberá utilizar para desplazarse por el tótem, encontramos:

- **Seleccionar:** Desplazar la mano derecha (predeterminada) con la palma abierta (mirando al sensor) y, una vez colocado el cursor encima del objeto de selección, pellizcar.
- **Retroceder al menú inicial:** Rápido movimiento de la mano derecha (predeterminada) con la palma abierta desde el frente del usuario hacia él.
- **Retroceder al anterior planeta:** Rápido movimiento de la mano derecha (predeterminada) con la palma abierta hacia la izquierda del usuario.
- **Avanzar al siguiente planeta:** Rápido movimiento de la mano derecha (predeterminada) con la palma abierta hacia la derecha del usuario.
- **Activar Modo Normal:** levantando los cinco dedos de la mano izquierda.
- **Activar Modo Niños:** levantando los dedos corazón e índice de la mano izquierda.
- **Activar Modo Experto:** levantando los dedos anular, corazón y meñique de la mano izquierda.

Una de las medidas de accesibilidad que tendrá nuestro tótem, aunque aún no se encuentre implementada, es la posibilidad de cambiar la mano dominante del usuario, de forma que se inviertan las manos en los gestos descritos previamente y facilitando así la exploración del tótem a las personas zurdas.

Por último, se ha implementado un gesto “oculto” conocido solo por el personal del museo. Consiste en levantar ambas palmas hacia arriba para permitir el cierre seguro de la aplicación del tótem. A través de Leap Motion se analiza en cada instante el vector normal de la palma de las dos manos y se comprueba que estén orientadas claramente hacia arriba. Si el sistema detecta simultáneamente las dos palmas en esta postura y el usuario la mantiene de forma continua durante 6 segundos, se interpreta como una orden intencionada de apagado.

4. Reparto de Trabajo

Santiago López Cerro	Diseño y modelado de los tótem no implementados y de los mapas usados. Búsqueda y recopilación de la información asociada a la memoria descriptiva junto con su posterior implementación.
Mario Líndez Martínez	Implementación del sistema de localización multilenguaje. Integración de los tutoriales en el proyecto. Implementación de parte de la interfaz gráfica en Unity. Desarrollo de la lógica interna asociada a distintos sistemas del proyecto.
Antonio Javier Rodríguez Romero	Búsqueda de información y realización de la memoria descriptiva del museo. Descripción de los <i>assets</i> y gestos utilizados para el tótem en la memoria

	técnica.
Diego Ortega Sanchez	Modelado 3D del Sistema Solar, junto con la implementación de los satélites y su información descriptiva adecuada a cada nivel.
Jaime Corzo Galdó	Incorporación de los distintos tipos de visitas, junto con los gestos asociados. Modelado del mapa 3D y de la idea conceptual de la guía durante el museo. Descripción del grafo de escena en la memoria técnica.

La implementación y desarrollo de los gestos ha sido realizada de forma colaborativa. Durante el proceso de desarrollo han sido numerosas las reuniones presenciales que hemos tenido para gestionar las futuras implementaciones y revisar los distintos avances que cada uno ha ido realizando por su cuenta. Por esto mismo consideramos que la carga de trabajo ha sido repartida de forma equitativa, habiendo así, trabajado todos por igual.